



IIC2343 - Arquitectura de Computadores (I/2026)

Interrogación 2 - Parte 2 - Sección 1

Pauta de evaluación

Pregunta 2: Representación de Números Reales (6 ptos.)

- (a) (2 ptos.) Indique cuántos bits de precisión posee la representación de un número float del estándar IEEE754 y explique por qué.

Solución: La representación de floats del estándar IEEE754 cuenta con **24 bits de precisión**. Esto se debe a la inclusión del bit implícito, el que otorga un bit de precisión adicional a los 23 bits del significante. Se otorga **1 pto.** por señalar la cantidad correcta de bits de precisión y **1 pto.** por justificación.

- (b) (4 ptos.) En los siguientes incisos, se le entregará un par de números reales A , B en formato float del estándar IEEE754 con notación hexadecimal. En cada caso, debe interpretar sus valores y realizar las operaciones de suma o multiplicación solicitadas. Luego, debe indicar el valor del resultado de la operación y señalar si se pierde precisión al almacenarlo de vuelta como float del estándar IEEE754, indicando además cómo queda almacenado en este formato. Los valores deben presentarse en base binaria con notación científica normalizada.

- (2 ptos.) $A + B$. $A = 0xBF800000$, $B = 0x7F800008$.

Solución:

- $A = 1\ 01111111\ 00000000000000000000000b$.
 - **Signo:** $1b \rightarrow$ negativo.
 - **Exponente:** $01111111b \rightarrow 127 - 127 = 0$.
 - **Significante:** $00000000000000000000000b \rightarrow 1,0b$.
 - **Valor:** $-(1,0 \times 10^0)b = -1b = -1$.
- $B = 0\ 11111111\ 000000000000000000001000b$.
 - **Signo:** $0b \rightarrow$ positivo.
 - **Exponente:** $11111111b \rightarrow$ reservado para infinito y NaN.

- De lo anterior, se deduce que la operación a realizar es $-1 + \text{NaN}$. En este caso, no se evaluará la forma en la que se realice la suma; solo que su resultado sea correcto. No obstante, **no es necesario realizarla para deducir que el resultado es igual a NaN** y, por ende, no hay pérdida de precisión al almacenar el valor de vuelta como float. En este caso, en estricto rigor la representación en formato float del resultado podría o no ser igual a B ya que depende de la implementación, por lo que se considera correcto si deducen que el resultado es NaN independiente de la representación que utilicen.

Se otorgan **0.5 ptos.** por la interpretación correcta de A ; **0.5 ptos.** por la interpretación correcta de B ; **0.5 ptos.** por señalar el resultado correcto de la operación; y **0.5 ptos.** por indicar correctamente si se pierde precisión o no. Además, en este caso se acepta no realizar la operación si se deduce que la suma es por NaN, manteniendo el valor final de B .

- Solución:**

- De lo anterior, se deduce que la operación a realizar es -2×32 . En este caso, no se evaluará la forma en la que se realice la multiplicación; solo que su resultado sea correcto. Aquí se opta por seguir el algoritmo visto en clases: se suman los exponentes sin desfase y se llega al valor $1 + 5 = 6 = 110b$. Luego, se multiplican los significantes: $-1, 0 \times 1, 0 = -1, 0$. Finalmente, se tiene como resultado $-1, 0 \times 10^{110}b = -1000000b = -64$, que en formato float se almacena como $11000010100000000000000000000000b = 0xC2800000$. **No existe pérdida de precisión.**

Se otorgan **0.5 ptos.** por la interpretación correcta de A ; **0.5 ptos.** por la interpretación correcta de B ; **0.5 ptos.** por señalar el resultado correcto de la operación; y **0.5 ptos.** por indicar correctamente si se pierde precisión o no. Se acepta escribir el valor en potencias de 2.

Pregunta 3: Comunicación con dispositivos I/O (6 ptos.)

- (a) (1.5 ptos.) Explique el significado y uso del identificador que envía el controlador de interrupciones luego de recibir la señal INTA por parte de la CPU.

Solución: Al recibir la señal INTA, la CPU espera obtener el identificador asociado al **dispositivo cuya interrupción será atendida**. Este será enviado por el controlador de interrupciones. Luego, la CPU hace uso de este valor como índice del vector de interrupciones para obtener el puntero de la ISR del dispositivo atendido, permitiendo así la ejecución de su subrutina. Se otorgan **0.75 ptos.** por indicar la asociación entre el identificador y los dispositivos I/O; y **0.75 ptos.** por indicar su uso en la ejecución de la ISR deseada.

- (b) (1.5 ptos.) Indique qué diferencia existe entre una interrupción gatillada por un dispositivo I/O y una gatillada por *software*.

Solución: La **única** diferencia es la señalada en el enunciado: una se gatilla desde un dispositivo I/O, mientras que la otra se gatilla por *software* (ya sea por una excepción o por un *trap*). En ambos casos, la interrupción se lleva a cabo a través de la ejecución de una ISR cuyo puntero se almacena en el vector de interrupciones, y ambas cuentan con un identificador que envía el controlador de interrupciones al recibir la señal INTA por parte de la CPU. Se otorgan **0.75 ptos.** por señalar la fuente de la interrupción como diferencia, y **0.75 ptos.** por explicar por qué no se diferencian en nada más.

- (c) (3 ptos.) Suponga que decide, con el propósito de ahorrar tiempo, automatizar la preparación de café filtrado a partir de un molinillo y hervidor eléctricos conectados a un computador con ISA RISC-V. A continuación, una representación gráfica de su implementación:



Para la comunicación con los electrodomésticos, sus registros de 32 bits son mapeados en memoria desde la dirección 0x600DCAFE:

Offset	Nombre	Descripción
0x00	coffee_grinder_status	Registro estado molinillo
0x04	coffee_grinder_command	Registro comando molinillo
0x08	coffee_grinder_grind_size	Config. grosor molienda
0x0C	coffee_grinder_weight	Config. gramos a moler
0x10	kettle_status	Registro estado hervidor
0x14	kettle_command	Registro comando hervidor
0x18	kettle_temperature	Config. temperatura hervidor (°C)

Nombre	Descripción	Valor
coffee_grinder_status	Apagado	0
coffee_grinder_status	Encendido	1
coffee_grinder_status	Moliendo	2
kettle_status	Apagado	0
kettle_status	Encendido	1
kettle_status	Hirviendo	2
coffee_grinder_command	Encender	1
coffee_grinder_command	Apagar	255
coffee_grinder_command	Moler	128
kettle_command	Encender	1
kettle_command	Apagar	255
kettle_command	Hervir	128
coffee_grinder_grind_size	Fino	1
coffee_grinder_grind_size	Medio	2
coffee_grinder_grind_size	Grueso	3

Pensando en su preparación favorita, desarrolla el siguiente programa:

```
coffee_system:
# Carga 0 + 0x600DCAFE en t0
addi t0, zero, 0x600DCAFE
addi t2, zero, 0
addi t3, zero, 0
addi s1, zero, 1
addi s2, zero, 2
addi s3, zero, 128
addi s4, zero, 255
addi s5, zero, 30
addi s6, zero, 92

step_one:
    lw t1, 0(t0) # Carga Mem[t0 + 0] en t1
    # Salta a continue_step_one si t1 != s2
    bne t1, s2, continue_step_one
    jal ra, step_five # Llamado a step_five
    beq zero, zero, step_three
    continue_step_one:
        beq t1, s1, step_two
        sw s1, 4(t0) # Carga s1 en Mem[t0 + 4]
    step_two:
        sw s2, 8(t0)
        sw s5, 12(t0)
        sw s3, 4(t0)
        jal ra, step_five
    # Continúa en step_three -->
```

```
step_three:
    lw t1, 16(t0) # Carga Mem[t0 + 16] en t1
    # Salta a continue_step_three si t1 != s2
    bne t1, s2, continue_step_three
    jal ra, step_six
    beq zero, zero, end # Salto incondicional
    continue_step_three:
        beq t1, s1, step_four
        sw s1, 20(t0)
    step_four:
        sw s6, 24(t0)
        sw s3, 20(t0)
        jal ra, step_six
        beq zero, zero, end
    step_five:
        while_step_five:
            lw t1, 0(t0)
            beq t1, s2, while_step_five
            sw s4, 4(t0)
            jalr zero, 0(ra) # Retorno
    step_six:
        while_step_six:
            lw t1, 16(t0)
            beq t1, s2, while_step_six
            sw s4, 20(t0)
            jalr zero, 0(ra)
    end: # Término del programa.
```

Describa, basándose en las tablas provistas, las tareas que realiza el programa en cada *label* **step_***. Haga una descripción según los estados y comandos; no una explicación por cada instrucción del programa.

Solución:

- **step_one (0.5 ptos.):** Se revisa el estado del molinillo. Si está moliendo, se ejecuta **step_five** y se salta a **step_three**. Si está encendida, se salta a **step_two**. En otro caso, se enciende antes de continuar con **step_two**.
- **step_two (0.5 ptos.):** Se configura el molinillo con una molienda de grosor medio y un total de 30 gramos. Luego, empieza a moler y se ejecuta **step_five** antes de seguir a **step_three**.
- **step_three (0.5 ptos.):** Se revisa el estado del hervidor. Si está hirviendo, se ejecuta **step_six** y se termina el programa. Si está encendida, se salta a **step_four**. En otro caso, se enciende antes de continuar con **step_four**.
- **step_four (0.5 ptos.):** Se configura el hervidor con una temperatura de 92°C. Luego, empieza a hervir y se ejecuta **step_six** antes de terminar el programa.
- **step_five (0.5 ptos.):** Se revisa el estado del molinillo hasta que deje de moler.
- **step_six (0.5 ptos.):** Se revisa el estado del hervidor hasta que deje de hervir.

Para cada **step**, se otorga la mitad del puntaje si existe como máximo un error o detalle faltante. Si existen dos o más errores o detalles faltantes, no se otorga puntaje.